INFINITE LOOP DETECTION IN SOFTWARE

USING STATIC AND RUNTIME ANALYSIS

CLA-2 ASSESSMENT TASK 4

A COURSE ASSIGNMENT REPORT

submitted for the course

21CSC301T – FORMAL LANGUAGE AND AUTOMATA

Submitted by

SWATHI PALANISAMY (RA2411026050013)
KRISHNA PRIYA P S (RA2411026050029)
A S LEKSHMI PRIYA (RA2411026050068)

Under the guidance of

Dr. C. GUNASUNDARI
Assistant Professor, Department of AIML

III Year / V Semester / Section A

B.Tech. Computer Science and Engineering with Specialization in Artificial Intelligence and Machine Learning

SCHOOL OF COMPUTING
FACULTY OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
TIRUCHIRAPPALLI – 621105

NOVEMBER 2026

CERTIFICATE

This is to certify that the assignment report titled “Infinite Loop Detection in Software Using Static and Runtime Analysis”, submitted by **Swathi Palanisamy (RA2411026050013), Krishna Priya P S (RA2411026050029)** and **A S Lekshmi Priya (RA2411026050068)** to the School of Computing, SRM Institute of Science and Technology, Tiruchirappalli, as the team work for Assessment Task 4 of Continuous Learning Assessment – 2 of the course 21CSC301T – Formal Language and Automata, is a bona fide record of the work carried out by the students under the guidance of Dr. C. Gunasundari, Assistant Professor, Department of Artificial Intelligence and Machine Learning.

To the best of our knowledge, the contents of this report have not been submitted to any other university or institute for the award of any degree or diploma.

Dr. C. Gunasundari **Head of the Department**

Assistant Professor, Dept. of AIML

Course Faculty

Place: Tiruchirappalli

Date: November 2026

DECLARATION

We declare that the work presented in this report was carried out as a team for Assessment Task 4 of the Formal Language and Automata course. The computational implementation was developed and executed using Python in a notebook environment. The report describes the problem formulation, static-analysis method, runtime timeout method, loop simulations, Turing-machine experiments, comparative analysis and visual results contained in the submitted notebook and graph archive.

The report follows the structure of the supplied reference report while using the code, outputs and graphs from the submitted Task 4 notebook. External theoretical concepts are acknowledged in the references.

Swathi Palanisamy (RA2411026050013)

Krishna Priya P S (RA2411026050029)

A S Lekshmi Priya (RA2411026050068)

Date: November 2026

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. C. Gunasundari, Assistant Professor, Department of Artificial Intelligence and Machine Learning, for her guidance in the Formal Language and Automata course and for giving us an opportunity to connect automata and computability concepts with a practical software problem.

We thank the School of Computing, SRM Institute of Science and Technology, Tiruchirappalli, for providing the academic environment and resources required to complete this assessment. We also acknowledge the support of our classmates, friends and family members during implementation, testing and documentation.

ABSTRACT

Infinite loops are programs or program paths that continue execution without reaching a terminating state. Detecting such behaviour is closely connected to the Halting Problem, which asks whether a computation will eventually halt. This report presents a practical Task 4 implementation that combines runtime timeout analysis, simple static analysis of Python while-statements, bounded loop simulation and a small Turing-machine simulator.

The submitted notebook first defines normal, infinite, conditional and countdown programs and evaluates them with a two-second runtime timeout. It then represents the results in tabular and graphical form. A static analyzer based on Python's AST module identifies a literal while True loop without a break statement as a possible infinite loop. Bounded simulation further examines loop progress using a maximum iteration limit. Finally, two simple Turing machines are simulated: one reaches a HALT state and another continues until the configured step limit.

The experiments show that practical detectors can identify clear infinite-loop patterns and provide useful warnings, but a timeout or bounded run is not a universal proof of non-termination. The work therefore illustrates the distinction between practical software analysis and the theoretical undecidability of the general Halting Problem.

Keywords: Infinite Loop, Halting Problem, Static Analysis, Runtime Analysis, AST, Bounded Simulation, Turing Machine, Decidability

TABLE OF CONTENTS

Chapter 1 Introduction

1.1 Team and roles

1.2 Background

1.3 Organisation of the report

Chapter 2 Problem Definition and Formal Model

2.1 The computational problem

2.2 Formal model

Chapter 3 Decidability Analysis

3.1 General Halting Problem

3.2 Practical detection versus exact decision

Chapter 4 Restricted-Case Solution

4.1 Runtime timeout analysis

4.2 Static AST analysis

4.3 Bounded loop simulation

4.4 Turing-machine simulation

Chapter 5 Validation and Results

5.1 Runtime benchmark

5.2 Static-analysis test cases

5.3 Loop simulation

5.4 Static versus runtime comparison

5.5 Turing-machine results

5.6 Final comparison

Chapter 6 Practical Implications

Chapter 7 Individual Contribution Statements

Chapter 8 Notebook and Reproducibility

Chapter 9 Conclusion

References

LIST OF FIGURES

Fig. 1. Program termination analysis

Fig. 2. Improved program termination analysis

Fig. 3. Bounded loop simulation

Fig. 4. Loop execution for different input values

Fig. 5. Static analysis versus runtime analysis

Fig. 6. Turing-machine execution comparison

LIST OF TABLES

Table 1. Team members and task focus

Table 2. Mapping between the real-world problem and computational model

Table 3. Runtime benchmark results

Table 4. Static-analysis test cases

Table 5. Bounded loop simulation results

Table 6. Static versus runtime comparison

Table 7. Turing-machine results

Table 8. Final comparison of all test programs

Chapter 1 Introduction

Assessment Task 4 of CLA-2 connects a real-world computational problem with the concepts of Formal Language and Automata. The selected problem is infinite-loop detection in software. A program that fails to reach a terminating condition may consume CPU time, block an application or cause a service to remain unavailable.

The notebook approaches the problem through several complementary techniques. Runtime analysis executes a program under a fixed timeout. Static analysis inspects Python source code using the AST representation. Bounded simulation checks whether a loop reaches its limit within a maximum number of iterations. A Turing-machine simulator then connects the software problem directly to a formal model of computation.

1.1 Team and roles

Member	Register Number	Task focus
Swathi Palanisamy	RA2411026050013	Problem modelling, runtime analysis and implementation
Krishna Priya P S	RA2411026050029	Static analysis, loop simulation and validation
A S Lekshmi Priya	RA2411026050068	Turing-machine experiments, graphs and documentation

1.2 Background

The Halting Problem asks whether an arbitrary program or Turing machine will eventually stop on a given input. In general, this cannot be decided by one algorithm for every possible program. Practical software tools therefore work with restricted program patterns, finite bounds, heuristics, static rules or monitoring policies.

1.3 Organisation of the report

Chapter 2 defines the computational problem and maps it to a formal model. Chapter 3 explains the decidability limitation. Chapter 4 describes the four practical analysis components implemented in the notebook. Chapter 5 reports the actual test results and graphs. Chapters 6–9 discuss implications, contributions, reproducibility and conclusions.

Chapter 2 Problem Definition and Formal Model

2.1 The computational problem

Given a program P and an input I, determine whether the execution of P on I terminates or continues indefinitely. The notebook demonstrates this question using small Python programs whose behaviour is known from their loop conditions and update rules.

For runtime analysis, the program is executed in a separate process. If it finishes before the two-second timeout, the result is Terminated. If the process remains alive after the timeout, it is terminated and the notebook reports Possible Infinite Loop.

2.2 Formal model

At the theoretical level, the problem can be expressed using the language HALT = {⟨M,w⟩ : M halts on input w}. The corresponding non-halting set is LOOP = {⟨M,w⟩ : M runs forever on w}. The notebook does not claim to decide LOOP for arbitrary programs; it implements restricted practical checks.

Real-world concept	**Computational representation**
Python program P	Program or machine M
Program input	Initial machine/program state
One execution	Sequence of configurations or program states
Program terminates	Execution reaches a return or HALT state
Possible infinite loop	Execution does not finish within selected bound
Static warning	AST pattern matching
Runtime warning	Timeout-based process monitoring
Bounded simulation	Finite maximum iterations

Chapter 3 Decidability Analysis

3.1 General Halting Problem

For arbitrary programs, there is no general algorithm that always terminates and correctly decides whether every program will halt. This is the classical Halting Problem. The result follows from the standard contradiction/diagonal argument: assuming a perfect halting decider exists allows construction of a program that deliberately does the opposite of the decider's prediction on its own description.

Therefore, an exact universal infinite-loop detector cannot be implemented. A practical tool must either restrict the class of programs it analyzes, use a finite execution bound, or allow an uncertain result.

3.2 Practical detection versus exact decision

The notebook's runtime detector uses a two-second timeout. This is a useful operational policy but not a mathematical proof. A slow but correct program can exceed the timeout. Similarly, the bounded loop simulation returns Possible Infinite Loop after 100 iterations when the loop has not terminated; this indicates that the selected bound was reached.

The AST detector is more targeted. It checks for a while statement whose condition is the constant True and searches the loop subtree for a break. This can identify an obvious class of infinite loops, but source-code patterns outside that rule require further analysis.

Method	Decision scope	Limitation
Runtime timeout	Programs that finish within the time bound	Slow terminating programs may be flagged
AST static rule	Literal while True patterns	Cannot prove arbitrary termination
Bounded simulation	Loop models within max iterations	Finite bound cannot prove all non-termination
Turing-machine simulation	Configured transition systems	Step limit prevents unrestricted proof

Chapter 4 Restricted-Case Solution

4.1 Runtime timeout analysis

The notebook defines run_with_timeout(function, timeout=2). It launches the function in a separate process, waits for the configured timeout and checks whether the process is still alive. A completed process is classified as Terminated; a process still running after the timeout is terminated and classified as Possible Infinite Loop.

```
def run_with_timeout(function, timeout=2):
    process = multiprocessing.Process(target=function)
    process.start()
    process.join(timeout)

    if process.is_alive():
        process.terminate()
        process.join()
        return "Possible Infinite Loop"

    return "Terminated"


print("Normal Program:", run_with_timeout(normal_program))
print("Infinite Loop:", run_with_timeout(infinite_loop))
print("Conditional Loop:", run_with_timeout(conditional_loop))
print("Countdown:", run_with_timeout(countdown))
```

4.2 Static AST analysis

The detect_infinite_loop function parses source code with Python's ast module. It searches for ast.While nodes. If the loop condition is the constant True and no break statement is found in the loop subtree, the result is Possible Infinite Loop. If a break is found, the function returns Needs Further Analysis.

```
import ast

def detect_infinite_loop(code):
    tree = ast.parse(code)

    for node in ast.walk(tree):
        if isinstance(node, ast.While):
            if isinstance(node.test, ast.Constant) and node.test.value is True:
                has_break = any(
                    isinstance(child, ast.Break)
                    for child in ast.walk(node)
                )

                if not has_break:
                    return "Possible Infinite Loop"

                return "Needs Further Analysis"

    return "No Obvious Infinite Loop Detected"
```

```
test_programs = {
    "Program 1": """
while True:
    print("Running")
""",
    "Program 2": """
i = 0
while i < 5:
    i += 1
""",
    "Program 3": """
while True:
    break
"""
}

for name, code in test_programs.items():
    print(name, ":", detect_infinite_loop(code))
```

4.3 Bounded loop simulation

The simulate_loop function uses start, limit and step values. Each iteration increments the counter by step. If the maximum number of iterations is reached before count becomes greater than or equal to limit, the function reports Possible Infinite Loop. Otherwise it reports Terminated.

```
def simulate_loop(start, limit, step, max_iterations=100):
    count = start
    iterations = 0

    while count < limit:
        if iterations >= max_iterations:
            return "Possible Infinite Loop", iterations

        count += step
        iterations += 1

    return "Terminated", iterations


test_cases = [
    ("Finite Loop", 0, 10, 1),
    ("Infinite Loop", 0, 10, 0),
    ("Another Finite Loop", 5, 20, 2)
]

results = []

for name, start, limit, step in test_cases:
    status, iterations = simulate_loop(start, limit, step)

    results.append([name, status, iterations])
```

```
df_simulation = pd.DataFrame(
    results,
    columns=["Program", "Execution Status", "Iterations"]
)

print(df_simulation.to_string(index=False))
```

4.4 Turing-machine simulation

The notebook also implements a simple Turing-machine simulator. A tape is stored as a dictionary, the head position is updated according to transition rules, and execution continues until HALT or the maximum step count is reached. This connects the practical software problem to the formal computation model.

```
# Simple Turing Machine Simulator

def run_turing_machine(tape_input, transitions, start_state="q0",
                       halt_state="HALT", max_steps=20):

    tape = {i: symbol for i, symbol in enumerate(tape_input)}
    head = 0
    state = start_state
    steps = 0

    while state != halt_state and steps < max_steps:
        symbol = tape.get(head, "_")

        if (state, symbol) not in transitions:
            break

        new_state, write_symbol, move = transitions[(state, symbol)]

        tape[head] = write_symbol
        state = new_state
        head += move
        steps += 1

    if state == halt_state:
        return "Machine Halted", steps

    return "Did Not Halt Within Step Limit", steps


# Machine 1: Moves right and halts at the blank symbol
halting_machine = {
    ("q0", "1"): ("q0", "1", 1),
    ("q0", "_"): ("HALT", "_", 0)
}

# Machine 2: Continues moving right forever
non_halting_machine = {
    ("q0", "1"): ("q0", "1", 1),
```

```
    ("q0", "_"): ("q0", "_", 1)
}

print("Halting Machine:",
      run_turing_machine("111", halting_machine))

print("Non-Halting Machine:",
      run_turing_machine("111", non_halting_machine))
```

Chapter 5 Validation and Results

5.1 Runtime benchmark

Four programs were evaluated with the two-second timeout: Normal Program, Infinite Loop, Conditional Loop and Countdown. The normal program sums the values 0 through 4 and returns 10. The conditional loop reaches 5, while the countdown reaches 0.

```
def normal_program():
    total = 0
    for i in range(5):
        total += i
    return total


def infinite_loop():
    while True:
        pass


def conditional_loop():
    count = 0
    while count < 5:
        count += 1
    return count


def countdown():
    n = 5
    while n > 0:
        n -= 1
    return n


print("Normal Program Output:", normal_program())
print("Conditional Loop Output:", conditional_loop())
print("Countdown Output:", countdown())
def run_with_timeout(function, timeout=2):
    process = multiprocessing.Process(target=function)
    process.start()
    process.join(timeout)

    if process.is_alive():
        process.terminate()
        process.join()
        return "Possible Infinite Loop"

    return "Terminated"


print("Normal Program:", run_with_timeout(normal_program))
```

```
print("Infinite Loop:", run_with_timeout(infinite_loop))
print("Conditional Loop:", run_with_timeout(conditional_loop))
print("Countdown:", run_with_timeout(countdown))
```

Program	Expected behaviour	Observed status
Normal Program	Terminates	Terminated
Infinite Loop	Runs forever	Possible Infinite Loop
Conditional Loop	Terminates	Terminated
Countdown	Terminates	Terminated

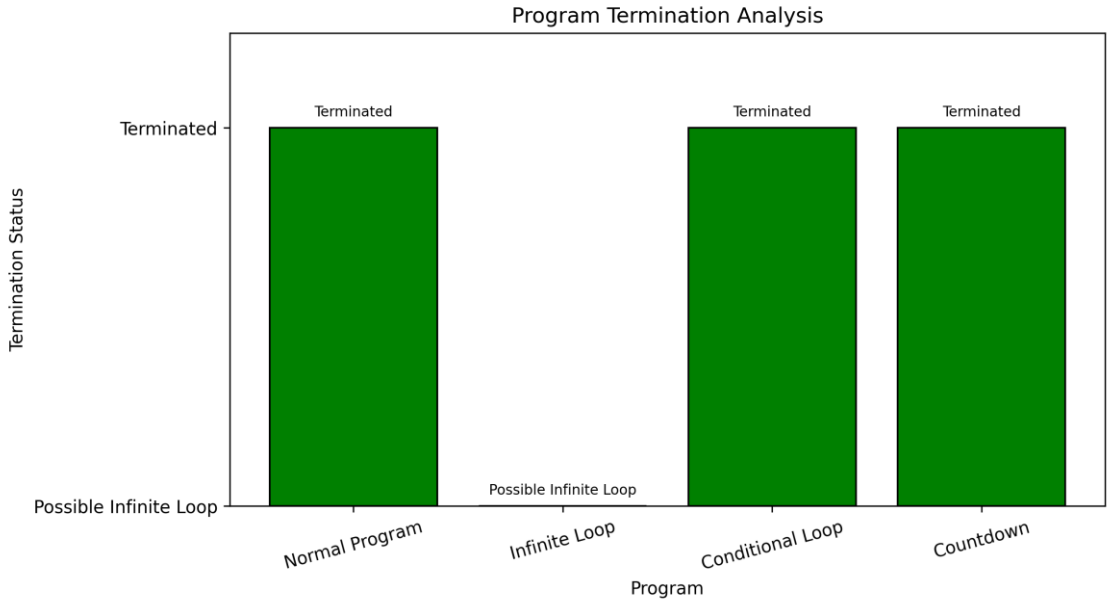


Fig. 1. Program Termination Analysis

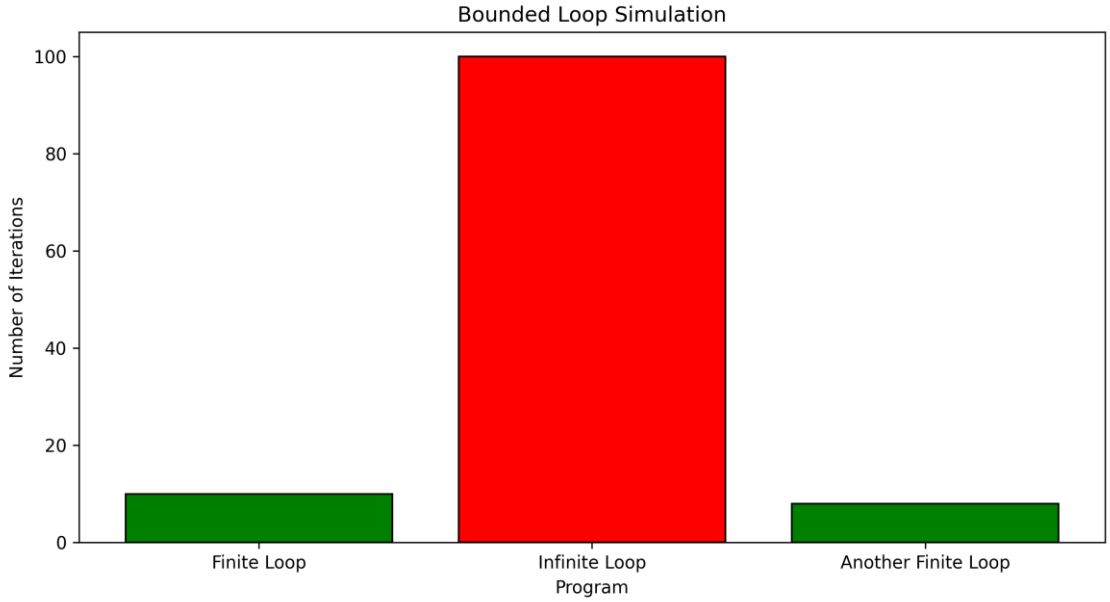


Fig. 2. Bounded Loop Simulation

Chapter 5 Validation and Results (continued)

5.2 Static-analysis test cases

```
import ast

def detect_infinite_loop(code):
    tree = ast.parse(code)

    for node in ast.walk(tree):
        if isinstance(node, ast.While):
            if isinstance(node.test, ast.Constant) and node.test.value is True:
                has_break = any(
                    isinstance(child, ast.Break)
                    for child in ast.walk(node)
                )

                if not has_break:
                    return "Possible Infinite Loop"

                return "Needs Further Analysis"

    return "No Obvious Infinite Loop Detected"


test_programs = {
    "Program 1": """
while True:
    print("Running")
""",
    "Program 2": """
i = 0
while i < 5:
    i += 1
""",
    "Program 3": """
while True:
    break
"""
}

for name, code in test_programs.items():
    print(name, ":", detect_infinite_loop(code))
```

Program	Static result
Program 1 – while True without break	Possible Infinite Loop
Program 2 – counter loop	No Obvious Infinite Loop Detected
Program 3 – while True with break	Needs Further Analysis

5.3 Bounded loop simulation

```
def simulate_loop(start, limit, step, max_iterations=100):
    count = start
    iterations = 0

    while count < limit:
        if iterations >= max_iterations:
            return "Possible Infinite Loop", iterations

        count += step
        iterations += 1

    return "Terminated", iterations


test_cases = [
    ("Finite Loop", 0, 10, 1),
    ("Infinite Loop", 0, 10, 0),
    ("Another Finite Loop", 5, 20, 2)
]

results = []

for name, start, limit, step in test_cases:
    status, iterations = simulate_loop(start, limit, step)

    results.append([name, status, iterations])

df_simulation = pd.DataFrame(
    results,
    columns=["Program", "Execution Status", "Iterations"]
)

print(df_simulation.to_string(index=False))
```

Program	Execution status	Iterations
Finite Loop	Terminated	10
Infinite Loop	Possible Infinite Loop	100
Another Finite Loop	Terminated	8

5.4 Input-value experiment

```
# Test different loop conditions

def test_loop_conditions():
    test_values = [0, 1, 5, 10, 20]

    results = []

    for value in test_values:
```

```
        count = 0

        while count < value:
            count += 1

        results.append({
            "Input": value,
            "Final Count": count,
            "Status": "Terminated"
        })

    return pd.DataFrame(results)


df_conditions = test_loop_conditions()
print(df_conditions.to_string(index=False))
```

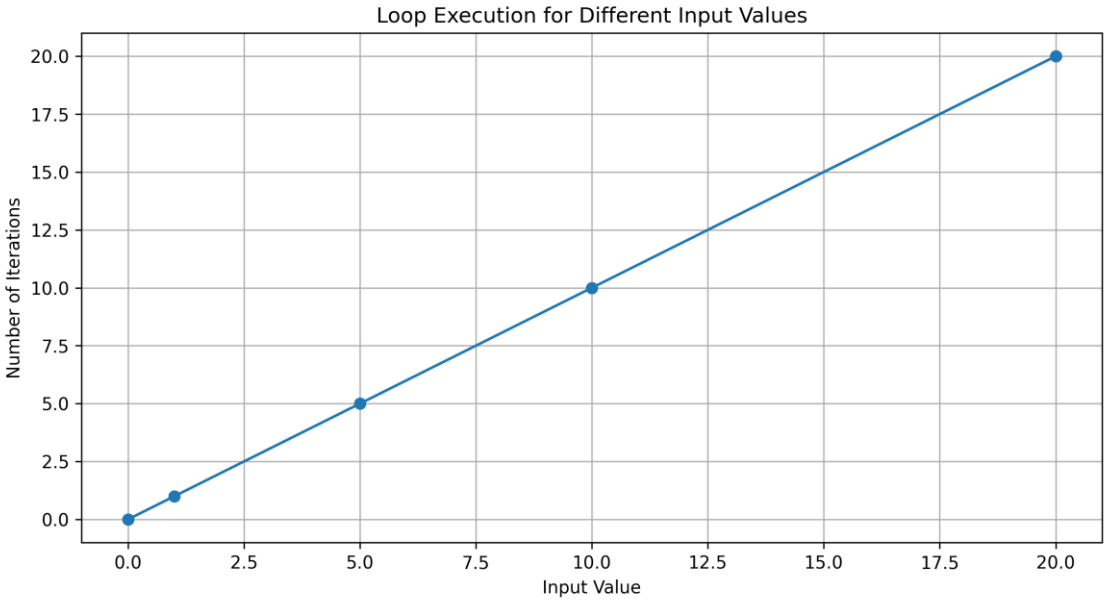


Fig. 3. Loop Execution for Different Input Values

Chapter 5 Validation and Results (continued)

5.5 Static versus runtime analysis

```
# Compare Static Analysis and Runtime Analysis

comparison = {
    "Program": [
        "Normal Program",
        "Infinite Loop",
        "Conditional Loop",
        "Constant True Loop"
    ],
    "Static Analysis": [
        "No Obvious Loop",
        "Possible Infinite Loop",
        "No Obvious Infinite Loop",
        "Possible Infinite Loop"
    ],
    "Runtime Analysis": [
        "Terminated",
        "Possible Infinite Loop",
        "Terminated",
        "Possible Infinite Loop"
    ]
}

df_comparison = pd.DataFrame(comparison)

print(df_comparison.to_string(index=False))
```

Program	Static Analysis	Runtime Analysis
Normal Program	No Obvious Loop	Terminated
Infinite Loop	Possible Infinite Loop	Possible Infinite Loop
Conditional Loop	No Obvious Infinite Loop	Terminated
Constant True Loop	Possible Infinite Loop	Possible Infinite Loop

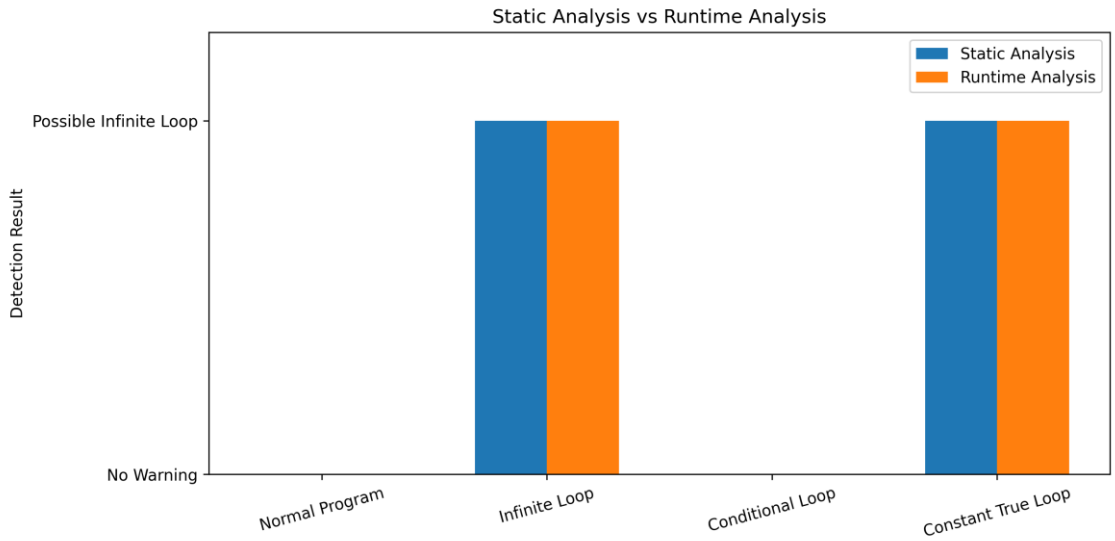


Fig. 4. Static Analysis vs Runtime Analysis

5.6 Turing-machine results

```
# Simple Turing Machine Simulator

def run_turing_machine(tape_input, transitions, start_state="q0",
                       halt_state="HALT", max_steps=20):

    tape = {i: symbol for i, symbol in enumerate(tape_input)}
    head = 0
    state = start_state
    steps = 0

    while state != halt_state and steps < max_steps:
        symbol = tape.get(head, "_")

        if (state, symbol) not in transitions:
            break

        new_state, write_symbol, move = transitions[(state, symbol)]

        tape[head] = write_symbol
        state = new_state
        head += move
        steps += 1

    if state == halt_state:
        return "Machine Halted", steps

    return "Did Not Halt Within Step Limit", steps


# Machine 1: Moves right and halts at the blank symbol
halting_machine = {
```

```
    ("q0", "1"): ("q0", "1", 1),
    ("q0", "_"): ("HALT", "_", 0)
}

# Machine 2: Continues moving right forever
non_halting_machine = {
    ("q0", "1"): ("q0", "1", 1),
    ("q0", "_"): ("q0", "_", 1)
}

print("Halting Machine:",
      run_turing_machine("111", halting_machine))

print("Non-Halting Machine:",
      run_turing_machine("111", non_halting_machine))
```

Machine	Expected behaviour	Observed result
Halting Machine	Reach HALT state	Machine Halted – 4 steps
Non-Halting Machine	Continue execution	Did Not Halt Within Step Limit – 20 steps

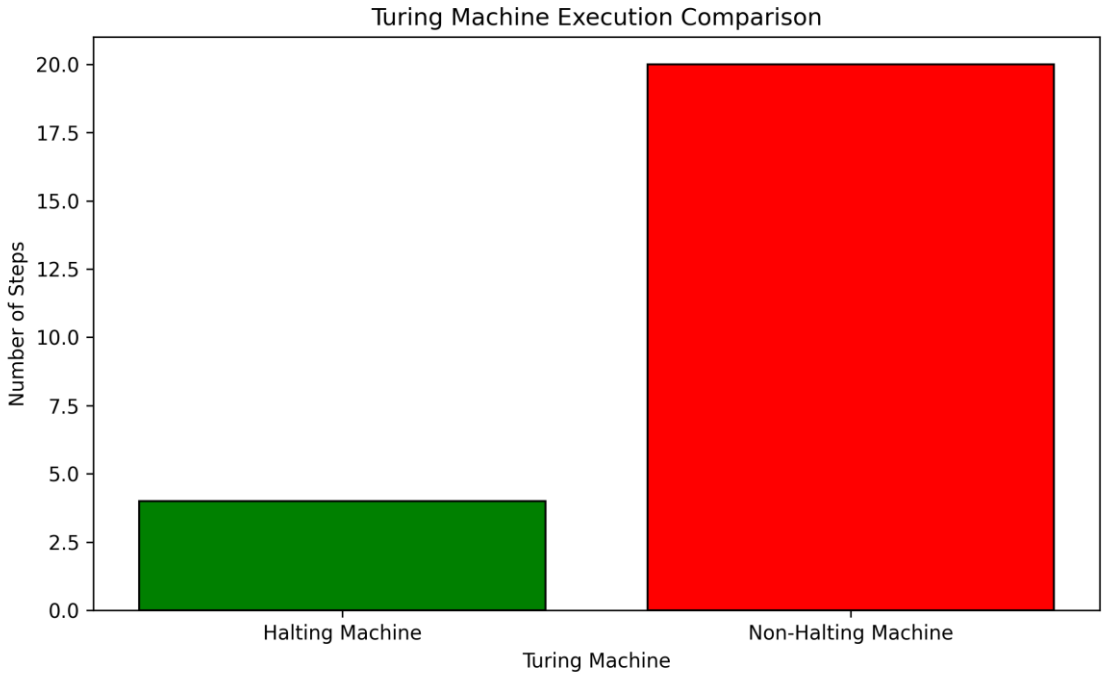


Fig. 5. Turing Machine Execution Comparison

Chapter 5 Validation and Results (continued)

5.7 Final comparison of all test programs

```
# Final comparison of all test programs

final_results = pd.DataFrame({
    "Test Case": [
        "Normal Program",
        "Infinite Loop",
        "Conditional Loop",
        "Countdown",
        "Halting Turing Machine",
        "Non-Halting Turing Machine"
    ],
    "Expected Behaviour": [
        "Terminates",
        "Runs Forever",
        "Terminates",
        "Terminates",
        "Reaches HALT State",
        "Continues Execution"
    ],
    "Observed Result": [
        run_with_timeout(normal_program),
        run_with_timeout(infinite_loop),
        run_with_timeout(conditional_loop),
        run_with_timeout(countdown),
        run_turing_machine("111", halting_machine)[0],
        run_turing_machine("111", non_halting_machine)[0]
    ]
})

print(final_results.to_string(index=False))
```

Test Case	Expected Behaviour	Observed Result
Normal Program	Terminates	Terminated
Infinite Loop	Runs Forever	Possible Infinite Loop
Conditional Loop	Terminates	Terminated
Countdown	Terminates	Terminated
Halting Turing Machine	Reaches HALT State	Machine Halted
Non-Halting Turing Machine	Continues Execution	Did Not Halt Within Step Limit

The final comparison confirms that the runtime tests, static checks and Turing-machine experiments provide consistent results for the selected restricted cases. The notebook also exports these final execution results to infinite_loop_detection_results.csv.

5.8 Overall findings

The experiments demonstrate three important observations. First, clear terminating programs can be identified by direct execution. Second, obvious infinite-loop patterns such as while True without break can be detected statically. Third, a timeout or finite step limit can identify executions that fail to terminate within the selected bound, but it cannot establish a universal proof of non-termination.

Chapter 6 Practical Implications

Infinite-loop detection is useful in testing, debugging, server monitoring, embedded systems and automated code-quality tools. A runtime timeout can protect a service from an execution that does not complete within an acceptable time. Static analysis can detect obvious patterns before execution and therefore provide earlier feedback to programmers.

The notebook also demonstrates why multiple techniques are useful together. Static analysis can be fast and preventive but has limited pattern coverage. Runtime analysis observes actual behaviour but requires execution and a policy for the timeout. Bounded simulation gives controlled experiments, while the Turing-machine simulator provides the theoretical bridge to formal computation.

A key limitation is that the notebook uses simple program patterns. The AST detector does not analyze arbitrary arithmetic progress, nested semantic conditions or all possible break paths. The runtime timeout can classify a slow terminating program as a possible infinite loop. The Turing-machine experiment is likewise bounded by max_steps.

Future improvements could include richer control-flow analysis, symbolic execution, ranking functions, loop invariants, static analysis of more Python constructs, configurable timeouts and finite-state model checking for bounded hardware models.

Chapter 7 Individual Contribution Statements

Swathi Palanisamy (RA2411026050013), problem modelling and runtime analysis. Contributed to the problem formulation, runtime timeout implementation and execution-status validation.

Krishna Priya P S (RA2411026050029), static and bounded analysis. Contributed to AST-based infinite-loop detection, bounded loop simulation and comparative result analysis.

A S Lekshmi Priya (RA2411026050068), Turing-machine analysis and documentation. Contributed to the Turing-machine simulator, execution graphs and report documentation.

All three members reviewed the test results and final report.

Chapter 8 Notebook and Reproducibility

The computational work is contained in the submitted notebook Untitled1.ipynb. The notebook contains 17 code cells covering program definitions, runtime analysis, tabular results, termination graphs, AST-based static analysis, bounded loop simulation, input-value testing, static-versus-runtime comparison, Turing-machine simulation, step-limit comparison, final validation and CSV export.

8.1 Notebook execution sequence

Stage	Notebook activity
1	Import time, multiprocessing, pandas and matplotlib
2	Define normal, infinite, conditional and countdown programs
3	Run each program with a two-second timeout
4	Create runtime result table and termination graph
5	Create improved termination visualization
6	Run AST-based static analysis
7	Run bounded loop simulation
8	Plot bounded simulation results
9–10	Test different loop input values and plot results
11–12	Compare static and runtime analysis and plot comparison
13–14	Simulate halting/non-halting Turing machines and compare step limits
15	Generate final comparison table
16	Export final execution results to CSV

8.2 Reproduction procedure

1. Open Google Colab or a compatible Jupyter environment.
2. Upload Untitled1.ipynb.
3. Run the notebook cells in sequence from top to bottom.
4. Verify the printed tables and the five supplied graph images.
5. Confirm that the final CSV file is generated as infinite_loop_detection_results.csv.

The graph archive supplied with the task contains the five figures used in this report: Program_Termination_Analysis.png, Bounded_Loop_Simulation.png, Loop_Execution_Input_Values.png, Static_vs_Runtime_Analysis.png and Turing_Machine_Comparison.png.

Chapter 9 Conclusion

This report presented a practical study of infinite-loop detection using the concepts of Formal Language and Automata. The submitted notebook implemented runtime timeout analysis, AST-based static analysis, bounded loop simulation and a simple Turing-machine simulator.

The runtime experiments correctly classified the selected normal, conditional and countdown programs as terminated and identified the deliberately infinite loop as a possible infinite loop. The AST analysis identified an obvious while True pattern without break, while the bounded simulation demonstrated how a finite iteration limit can expose a loop that makes no progress. The Turing-machine experiment further connected the practical problem to the formal notion of halting.

The work also highlights the theoretical limitation behind the problem. No general algorithm can decide termination for every arbitrary program. Consequently, the practical results in this notebook must be interpreted within their stated restrictions and bounds.

Overall, the task demonstrates how theoretical concepts such as the Halting Problem and Turing machines can be connected to practical software testing and program analysis. The combined approach is useful for identifying clear cases while remaining honest about the limits of general termination detection.

REFERENCES

[1] M. Sipser, Introduction to the Theory of Computation, 3rd ed., Cengage Learning, 2013.

[2] J. E. Hopcroft, R. Motwani and J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 3rd ed., Pearson, 2007.

[3] A. M. Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem,” Proceedings of the London Mathematical Society, 1937.

[4] H. G. Rice, “Classes of Recursively Enumerable Sets and Their Decision Problems,” Transactions of the American Mathematical Society, vol. 74, no. 2, 1953.

[5] Python Software Foundation, Python 3 Documentation, Python AST and multiprocessing modules.

[6] Google Colab, hosted Jupyter notebook environment used for the computational implementation.

[7] Submitted notebook, Untitled1.ipynb, Assessment Task 4 implementation, 2026.

[8] Submitted graph archive, Task4_All_Graphs.zip, five generated figures, 2026.